

TP : Système de notation pour un tournoi d'escrime fantastique

Contexte du projet

Vous développez le backend d'un jeu de rôle fantastique où des chevaliers, paladins et autres héros participent à des tournois d'escrime épiques. Ces tournois suivent des règles de notation complexes qui déterminent le champion final.

Imaginez une arène magique où Sir Galahad affronte Dame Morgane, puis combat le Chevalier Noir, etc. Chaque duel a un résultat (victoire, match nul, ou défaite) et le système doit calculer automatiquement le score final de chaque participant selon des règles très précises.

Au-delà du calcul, vous devrez construire un système complet dans un second temps : persistance des joueurs en base de données, API REST pour interagir avec le système, notification des participants et une application web, et bien sûr, l'ensemble couvert par des tests à tous les niveaux.

Objectifs pédagogiques

À la fin de ce TP, vous saurez :

- **Structurer des tests unitaires** avec xUnit de manière claire et organisée
- **Utiliser FluentAssertions** pour des assertions plus lisibles et expressives
- **Tester des scénarios métier complexes** avec règles de calcul
- **Isoler des dépendances avec Moq** pour tester un service de manière unitaire.
- **Gérer les cas limites** et les exceptions dans vos tests
- **Tester des collections** et des objets complexes
- **Appliquer les bonnes pratiques** de nommage et d'organisation des tests
- **Configurer un pipeline GitHub Actions** complet avec couverture de code.

Règles du tournoi (à implémenter)

Système de points de base

1. **Victoire** : +3 points
2. **Match nul** : +1 point
3. **Défaite** : 0 point

Règles spéciales

4. **Bonus de série** : +5 points si un joueur remporte **3 combats consécutifs ou plus**
 - Win-Win-Win → Bonus accordé
 - Win-Win-Win-Win → Bonus accordé (une seule fois)
 - Win-Loss-Win-Win-Win → Bonus accordé seulement pour les 3 dernières victoires
5. **Disqualification** : Score total = 0, peu importe les performances passées
 - Exemple : Un joueur avait 15 points, mais il triche → Score final = 0
6. **Pénalités** : Points négatifs soustraits du score, mais le score final ne peut jamais être négatif
 - Exemple : Score = 8, Pénalités = 10 → Score final = 0 (pas -2)

Structure du code à implémenter

Classe MatchResult

```
public class MatchResult
{
    public enum Result
    {
        Win,      // Victoire
        Draw,     // Match nul
        Loss      // Défaite
    }

    public Result Outcome { get; set; }

    // Constructeur pour faciliter les tests
    public MatchResult(Result outcome)
    {
        Outcome = outcome;
    }

    // Constructeur par défaut
    public MatchResult() { }
}
```

Classe ScoreCalculator (à implémenter)

```
public class ScoreCalculator
{
    /// <summary>
    /// Calcule le score final d'un joueur selon les règles du tournoi
```

```

    /// </summary>
    /// <param name="matches">Liste des résultats de combat dans l'ordre
    chronologique</param>
    /// <param name="isDisqualified">True si le joueur est
    disqualifié</param>
    /// <param name="penaltyPoints">Points de pénalité (nombre
    positif)</param>
    /// <returns>Score final (jamais négatif)</returns>
    public int CalculateScore(List<MatchResult> matches, bool
    isDisqualified = false, int penaltyPoints = 0)
    {
        // TODO: À implémenter selon les règles
        throw new NotImplementedException();
    }
}

```

Exemples détaillés de calcul

Exemple 1 : Calcul simple

Résultats : Win, Draw, Loss, Win

Calcul : $3 + 1 + 0 + 3 = 7$ points

Bonus : Aucun (pas 3 victoires consécutives)

Score final : 7 points

Exemple 2 : Avec bonus

Résultats : Win, Win, Win, Draw

Calcul : $3 + 3 + 3 + 1 = 10$ points

Bonus : +5 points (3 victoires consécutives)

Score final : 15 points

Exemple 3 : Bonus multiples

Résultats : Win, Win, Win, Loss, Win, Win, Win, Win

Calcul : $3+3+3+0+3+3+3+3 = 21$ points

Bonus : +5 points (première série de 3) + 5 points (deuxième série de 4)

Score final : 31 points

Exemple 4 : Avec pénalités

Résultats : Win, Win, Draw

Calcul : $3 + 3 + 1 = 7$ points

Pénalités : 3 points

Score final : 4 points

Exemple 5 : Pénalités supérieures au score

Résultats : Win, Draw

Calcul : $3 + 1 = 4$ points

Pénalités : 10 points

Score final : 0 points (pas de score négatif)

Tests à implémenter avec xUnit + FluentAssertions

Installation des packages

```
<PackageReference Include="Microsoft.NET.Test.Sdk" Version="17.8.0" />
<PackageReference Include="xunit" Version="2.6.1" />
<PackageReference Include="xunit.runner.visualstudio" Version="2.5.3" />
<PackageReference Include="FluentAssertions" Version="6.12.0" />
```

Structure de test recommandée

```
public class ScoreCalculatorTests
{
    private readonly ScoreCalculator _calculator;

    public ScoreCalculatorTests()
    {
        _calculator = new ScoreCalculator();
    }

    // Tests de base
    [Fact]
    public void Should_Calculate_Basic_Score_Without_Bonus() { }

    [Fact]
    public void Should_Add_Bonus_For_Three_Consecutive_Wins() { }

    // Tests avec données paramétrées
    [Theory]
    [InlineData(new[] { "Win", "Win", "Win" }, 14)] // 9 points + 5
    bonus
    [InlineData(new[] { "Win", "Draw", "Win" }, 7)] // 7 points, pas de
    bonus
    public void Should_Calculate_Score_Correctly(string[] results, int
```

```

expectedScore) { }

    // Tests des cas limites
    [Fact]
    public void Should_Return_Zero_When_Disqualified() { }

    [Fact]
    public void Should_Not_Allow_Negative_Final_Score() { }
}

```

Liste complète des tests à implémenter

Tests de base (fonctionnement normal)

1. **Calcul simple** : Win, Draw, Loss → 4 points
2. **Que des victoires** : Win, Win → 6 points
3. **Que des nuls** : Draw, Draw, Draw → 3 points
4. **Que des défaites** : Loss, Loss → 0 points

Tests du bonus de série

5. **Bonus minimum** : Win, Win, Win → 14 points (9 + 5)
6. **Bonus avec 4 victoires** : Win, Win, Win, Win → 17 points (12 + 5)
7. **Pas de bonus si interrompu** : Win, Win, Loss, Win → 6 points
8. **Bonus multiple** : Win×3, Loss, Win×4 → 26 points (21 + 5)
9. **Bonus NON accordé** : Win, Draw, Win, Win → 10 points

Tests de disqualification

10. **Disqualifié avec score positif** → 0 points
11. **Disqualifié sans combats** → 0 points

Tests des pénalités

12. **Pénalités normales** : Score 10, Pénalités 3 → 7 points
13. **Pénalités supérieures** : Score 5, Pénalités 8 → 0 points
14. **Pénalités égales** : Score 7, Pénalités 7 → 0 points

Tests des cas limites

15. **Liste vide** : Aucun combat → 0 points
16. **Liste null** → Exception ArgumentNullException
17. **Pénalités négatives** → Exception ArgumentException
18. **Très long tournoi** : 100 combats avec pattern complexe

Tests avec données paramétrées

- 19. **Theory avec InlineData** : Plusieurs scénarios en un test
- 20. **Theory avec MemberData** : Cas complexes depuis une méthode

Exemples de tests avec FluentAssertions

Test basique

```
[Fact]
public void Should_Calculate_Basic_Score_Without_Bonus()
{
    // Arrange
    var matches = new List<MatchResult>
    {
        new(MatchResult.Result.Win),    // 3 points
        new(MatchResult.Result.Draw),   // 1 point
        new(MatchResult.Result.Loss)    // 0 point
    };

    // Act
    var score = _calculator.CalculateScore(matches);

    // Assert
    score.Should().Be(4, "because 3+1+0 = 4 points without bonus");
}
```

Test avec bonus

```
[Fact]
public void Should_Add_Bonus_For_Three_Consecutive_Wins()
{
    // Arrange
    var matches = new List<MatchResult>
    {
        new(MatchResult.Result.Win),
        new(MatchResult.Result.Win),
        new(MatchResult.Result.Win),
        new(MatchResult.Result.Draw)
    };

    // Act
    var score = _calculator.CalculateScore(matches);

    // Assert
```

```
score.Should().Be(15, "because 3*3 + 1 + 5 bonus = 15 points");
}
```

Test paramétré

```
[Theory]
[InlineData(3, 0, 0, 14)] // 3 wins → 9 + 5 bonus
[InlineData(2, 1, 0, 7)]  // 2 wins, 1 draw → 7, no bonus
[InlineData(0, 0, 3, 0)]  // 3 losses → 0 points
public void Should_Calculate_Score_With_Different_Results(int wins, int
draws, int losses, int expected)
{
    // Arrange
    var matches = new List<MatchResult>();
    for (int i = 0; i < wins; i++)
        matches.Add(new MatchResult.Result.Win());
    for (int i = 0; i < draws; i++)
        matches.Add(new MatchResult.Result.Draw());
    for (int i = 0; i < losses; i++)
        matches.Add(new MatchResult.Result.Loss());

    // Act
    var score = _calculator.CalculateScore(matches);

    // Assert
    score.Should().Be(expected);
}
```

Test d'exception

```
[Fact]
public void Should_Throw_Exception_When_Matches_Is_Null()
{
    // Act & Assert
    Action act = () => _calculator.CalculateScore(null);

    act.Should().Throw<ArgumentNullException>()
        .WithParameterName("matches")
        .WithMessage("*cannot be null*");
}
```

Livrables Partie 1

- ScoreCalculator.cs entièrement implémenté.

- ScoreCalculatorTests.cs avec au minimum 15 tests couvrant tous les scénarios ci-dessus.
- Couverture de code de ScoreCalculator $\geq 95\%$.
- Tous les tests verts.

Système de classement

Si vous finissez rapidement, implémentez une classe `TournamentRanking` :

```
public class Player
{
    public string Name { get; set; }
    public List<MatchResult> Matches { get; set; } = new();
    public bool IsDisqualified { get; set; }
    public int PenaltyPoints { get; set; }
}

public class TournamentRanking
{
    private readonly ScoreCalculator _scoreCalculator;

    public TournamentRanking(ScoreCalculator scoreCalculator)
    {
        _scoreCalculator = scoreCalculator;
    }

    /// <summary>
    /// Classe les joueurs par score décroissant
    /// </summary>
    public List<Player> GetRanking(List<Player> players)
    {
        // TODO: Implémenter le classement
    }

    /// <summary>
    /// Trouve le champion (joueur avec le meilleur score)
    /// </summary>
    public Player GetChampion(List<Player> players)
    {
        // TODO: Implémenter
    }
}
```

Tests à implémenter

- Classement correct par score décroissant
- Gestion des égalités (même score)
- Champion avec score maximum
- Cas où tous les joueurs sont disqualifiés

Evolution & Intégration dans un CI/CD

- Mettez en place une CI complète sur GitHub Actions qui exécute tous vos tests à chaque push et publie la couverture de code.
- Rédigez un plan de test documenté et intégrez aux tests
- Faites évoluer l'application en créant une base de données , une application API, une application web interactive pour jouer le jeu.

Critères d'évaluation

Qualité du code (40%)

- Logique de calcul correcte
- Gestion des cas limites
- Gestion des exceptions
- Code lisible et bien structuré

Qualité des tests (40%)

- Couverture complète des scénarios
- Tests bien nommés et organisés
- Utilisation appropriée de FluentAssertions
- Tests paramétrés quand approprié
- Isolation des tests (pas de dépendances)

Bonnes pratiques (20%)

- Structure AAA (Arrange, Act, Assert)
- Un seul concept testé par test
- Messages d'assertion explicites
- Pas de logique complexe dans les tests

Conseils de nommage

- `Should_Calculate_Correct_Score_With_Three_Wins()`
- `Should_Return_Zero_When_Player_Is_Disqualified()`
- `Test1()` ou `CalculateScoreTest()`